# ЛЕКЦИЯ 1. ВВЕДЕНИЕ В JAVA И ОСНОВЫ ANDROID-РАЗРАБОТКИ

Нативная разработка под Android (Java + Kotlin)

# СТРУКТУРА КУРСА

## НАТИВНАЯ РАЗРАБОТКА (3 ЛЕКЦИИ)

- Лекция 1: Java, JVM, Kotlin
- Лекция 2: Экосистема Android
- Лекция 3: Компоненты приложений

## КРОССПЛАТФОРМЕННАЯ (1 ЛЕКЦИЯ)

- Лекция 4: Обзор технологий

## FLUTTER (3 ЛЕКЦИИ)

- Лекция 5: Основы и виджеты
- Лекция 6: Навигация и состояние
- Лекция 7: Продвинутые темы

# СТРУКТУРА КУРСА. ПРАКТИКА

- 5 лабораторных работ (сентябрь-ноябрь).
- Для экзамена: Проект (декабрь-январь).
- Проект: приложение на выбор (нативное Android, Flutter или кроссплатформенное).

# МОБИЛЬНЫЕ ПЛАТФОРМЫ

## РЫНОК (2025)

- **Android**: 70-75% мирового рынка
- **iOS**: 25-30%
- Android: ~3.5 млн приложений (Google Play)
- iOS: ~1.8 млн приложений (App Store)

## ПОДХОДЫ К РАЗРАБОТКЕ

- **Нативная**: Java/Kotlin (Android), Swift (iOS)
- **Кроссплатформенная**: Flutter, React Native
- **Веб-приложения**: PWA

# ЯЗЫК JAVA: ОСНОВЫ

# JAVA: **ФИЛОСОФИЯ ЯЗЫКА**

- **"Write Once, Run Anywhere"** - основная концепция
- Объектно-ориентированный язык
- Статическая типизация
- Автоматическое управление памятью (Garbage Collection)
- Богатая стандартная библиотека
- Мультиплатформенность (JVM)

# ВЕРСИИ JAVA: ЭВОЛЮЦИЯ

## РАННИЕ ВЕРСИИ

- Java 1.0 (1996) - первая версия
- Java 1.2 (1998) - Collections Framework
- Java 5 (2004) - generics, аннотации, enum

## СОВРЕМЕННЫЕ ВЕРСИИ

- Java 8 (2014) - лямбды, Stream API (LTS)
- Java 11 (2018) - модули (LTS)
- Java 17 (2021) - sealed classes (LTS)
- Java 21 (2023) - virtual threads (LTS)
- Java 23 (2024) - latest

# JAVA ECOSYSTEM

## JRE

Java Runtime Environment

Для запуска приложений

## JDK

Java Development Kit

Для разработки (JRE + tools)

## JVM

Java Virtual Machine

Исполняющая среда

# СРАВНЕНИЕ JDK VS JRE VS JVM

| Компонент | Назначение | Включает | Для кого |
|---|---|---|---|
| **JVM** | Выполнение байт-кода | Исполняющий движок, память | Система |
| **JRE** | Запуск приложений | JVM + библиотеки + утилиты | Пользователи |
| **JDK** | Разработка приложений | JRE + компилятор + инструменты | Разработчики |

# ОСНОВНЫЕ КОМПОНЕНТЫ JDK

## КОМПИЛЯЦИЯ И ЗАПУСК

- **javac** - компиляция .java -> .class
- **java** - запуск приложений
- **jar** - упаковка в JAR-архив
- **javadoc** - генерация документации

## ОТЛАДКА И МОНИТОРИНГ

- **jdb** - отладчик
- **jstack** - дамп потоков
- **jmap** - дамп памяти
- **jconsole** - мониторинг JMX

# ПРОЦЕСС РАЗРАБОТКИ С JDK

1. **Написание кода** - создание .java файлов
2. **Компиляция** - javac -> .class файлы
3. **Упаковка** - jar -> JAR архив
4. **Запуск** - java -jar application.jar

```
# Компиляция
javac HelloWorld.java

# Запуск
java HelloWorld

# Создание JAR
jar cvf app.jar *.class
```

# JVM: ВИРТУАЛЬНАЯ МАШИНА JAVA

# JVM: **НАЗНАЧЕНИЕ**

**Виртуальная машина Java** - абстрактная вычислительная машина, обеспечивающая кроссплатформенность по принципу "Write Once, Run Anywhere".

- Загрузка и верификация байт-кода
- Выполнение программ
- Управление памятью (сборка мусора)
- Обеспечение безопасности
- Оптимизация производительности (JIT)

# АРХИТЕКТУРА JVM

## ОСНОВНЫЕ КОМПОНЕНТЫ

- **Class Loader Subsystem**
  - Loading
  - Linking (Verification, Preparation, Resolution)
  - Initialization
- **Runtime Data Areas**
  - Method Area (Metaspace)
  - Heap Area
  - Stack Area
  - PC Registers
  - Native Method Stack

## ИСПОЛНЯЮЩИЕ КОМПОНЕНТЫ

- **Execution Engine**
  - Interpreter
  - JIT Compiler
  - Garbage Collector
- **Native Method Interface** (JNI)
- **Native Method Libraries**

# ПОДСИСТЕМА ЗАГРУЗКИ КЛАССОВ: ДЕЛЕГИРОВАНИЕ

1. **Bootstrap ClassLoader** - загружает core библиотеки (java.lang, java.util)
2. **Platform ClassLoader** - загружает расширения платформы
3. **Application ClassLoader** - загружает классы приложения

Принцип делегирования: запрос сначала передаётся родительскому загрузчику.

# ФАЗЫ ЗАГРУЗКИ КЛАССОВ

## LOADING

- Bootstrap ClassLoader ищет класс в rt.jar
- Platform ClassLoader ищет в модулях платформы
- Application ClassLoader ищет в classpath
- Создаёт объект Class в Metaspace

## LINKING

- **Verification** - проверка байт-кода (bytecode verifier)
- **Preparation** - выделение памяти для static полей
- **Resolution** - замена символьных ссылок на прямые

Инициализация: выполнение static блоков и присвоение значений static полям.

# ПРИМЕР: ПРАВИЛО ДЕЛЕГИРОВАНИЯ

```
// При вызове: Class.forName("com.example.MyClass")

// Шаг 1: Application ClassLoader делегирует Platform
// Шаг 2: Platform ClassLoader делегирует Bootstrap
// Шаг 3: Bootstrap не находит -> Platform ищет
// Шаг 4: Platform не находит -> Application ищет в classpath

// Кастомный ClassLoader с переопределением:
public class CustomClassLoader extends ClassLoader {
    @Override
    protected Class<?> loadClass(String name, boolean resolve)
            throws ClassNotFoundException {
        // Сначала делегируем родителю
        Class<?> loadedClass = findLoadedClass(name);
        if (loadedClass == null) {
            try {
                loadedClass = getParent().loadClass(name);
```

```
            } catch (ClassNotFoundException e) {
                // Родитель не нашёл - ищем сами
                loadedClass = findClass(name);
            }
        }
        if (resolve) {
            resolveClass(loadedClass);
        }
        return loadedClass;
    }
}
```

# HEAP AREA: СТРУКТУРА ПАМЯТИ

## СТРУКТУРА КУЧИ

- **Young Generation**
  - Eden Space (новые объекты)
  - Survivor Space (S0, S1)
- **Old Generation** (Tenured)
- **Metaspace** (замена PermGen)

## ЖИЗНЕННЫЙ ЦИКЛ ОБЪЕКТА

1. Создание в Eden
2. Minor GC -> Survivor
3. После N копирований -> Old Generation
4. Major GC / Full GC

# ОБЛАСТИ ПАМЯТИ JVM ПОДРОБНО

| Область | Описание | Потокобезопасность | Размер |
|---|---|---|---|
| **Heap** | Хранит все объекты и массивы | Общий для всех потоков | -Xmx (до 256GB) |
| **Metaspace** | Метаданные классов (вместо PermGen) | Общий | -XX:MaxMetaspaceSize |
| **Stack** | Локальные переменные и вызовы методов | Каждый поток свой | -Xss (1MB по умолч.) |
| **PC Register** | Адрес текущей инструкции | Каждый поток свой | Размер слова CPU |

# STACK VS HEAP: КОГДА ЧТО ИСПОЛЬЗУЕТСЯ

## STACK (СТЕК)

- Локальные примитивы (int, long, double)
- Ссылки на объекты
- Вызовы методов (стек-фреймы)
- Автоматическое освобождение при выходе из метода
- Быстрое выделение/освобождение

## HEAP (КУЧА)

- Все объекты (new, arrays)
- String, Collections, custom classes
- Управление GC (сборка мусора)
- Доступ через ссылки со Stack
- Медленнее, но гибче

```
void calculate() {
    int x = 10;                 // Stack: примитив
    int[] arr = new int[5];  // Stack: ссылка, Heap: масси
    String s = new String("Hi"); // Stack: ссылка, Heap: с
}
```

# СБОРКА МУСОРА: ТИПЫ КОЛЛЕКТОРОВ

| Коллектор | Описание | Флаг |
|---|---|---|
| **Serial** | Один поток, для маленьких приложений | -XX:+UseSerialGC |
| **Parallel** | Множество потоков, максимальная пропускная способность | -XX:+UseParallelGC |
| **G1GC** | Баланс производительности и пропускной способности (по умолчанию) | -XX:+UseG1GC |
| **ZGC** | Минимальная задержка (<10ms) | -XX:+UseZGC |
| **Shenandoah** | Параллельная уборка с низкой задержкой | -XX:+UseShenandoahGC |

# GC-АЛГОРИТМЫ ПОДРОБНО

### SERIAL GC

- Один поток для Young и Old
- STW (Stop-The-World) при сборке
- Малый overhead

### PARALLEL GC

- Множество потоков для Young Gen
- Максимальная пропускная способность
- По умолчанию до Java 8

### G1 GC (GARBAGE FIRST)

- По умолчанию с Java 9
- Heap разделён на регионы
- Приоритетная сборка самых "мусорных"
- Предсказуемая задержка

### ZGC / SHENANDOAH

- Ultra-low latency (<10ms)
- Concurrent compaction
- Для серверных приложений

# КОГДА КАКОЙ GC ИСПОЛЬЗОВАТЬ

| Сценарий | Рекомендация | Причина |
|---|---|---|
| CLI утилита | Serial GC | Малый footprint, быстрое завершение |
| Batch-процессор | Parallel GC | Максимальный Throughput |
| Веб-сервер (малый) | G1 GC | Баланс latency/throughput |
| Микросервис | G1 GC | Предсказуемая задержка |
| Финансовые системы | ZGC | Минимальная задержка |
| HFT (высокочастотный трейдинг) | ZGC + Shenandoah | <1ms pauses |
| Android (ART Runtime) | ART GC (hybrid) | Concurrent mark-sweep + compaction |

# JIT-КОМПИЛЯЦИЯ

- **Just-In-Time** компиляция: "горячий" код компилируется в нативный
- Interpreter сначала выполняет байт-код
- Профайлер определяет "горячие" методы (hot spots)
- JIT компилирует их в нативный код и кэширует
- Производительность приближается к нативному коду

# JIT: МЕХАНИКА РАБОТЫ

## ЭТАПЫ JIT-КОМПИЛЯЦИИ

1. Interpreter выполняет байт-код
2. Method invocation counter увеличивается
3. При достижении порога (default: 10 000) - JIT compilation
4. C1 Compiler (baseline)
5. C2 Compiler (глубокие оптимизации)

## ОПТИМИЗАЦИИ JIT

- **Inlining** - подстановка тела метода
- **Escape Analysis** - оптимизация аллокаций
- **Loop Unrolling** - развёртывание циклов
- **Dead Code Elimination** - удаление мёртвого кода
- **Devirtualization** - замена виртуальных вызов

# TIERED COMPILATION: C1 VS C2

| **Характеристика** | **C1 (Client)** | **C2 (Server)** |
|---|---|---|
| Цель | Быстрая компиляция | Максимальная оптимизация |
| Глубина анализа | Мелкая | Глубокая (Graal) |
| Время компиляции | Малое | Большое |
| Профилирование | Простое | Детальное (inlining, unrolling) |
| Применение | Startup-фаза | Горячие методы |

By default: Tier 0 (interpret) -> Tier 1-3 (C1) -> Tier 4 (C2). Флаг: -XX:+TieredCompilation

# JAVA: GENERICS (ОБОБЩЕНИЯ)

```
// Без generics - необходимость кастов
List list = new ArrayList();
list.add("Hello");
String s = (String) list.get(0);  // unsafe cast

// С generics - type safety на этапе компиляции
List<String> names = new ArrayList<>();
names.add("Hello");
String s = names.get(0);  // без каста

// Ограничение типов (bounded type parameters)
public <T extends Comparable<T>> T max(T a, T b) {
    return a.compareTo(b) >= 0 ? a : b;
}

// Wildcards
List<? extends Number> numbers;  // чтение Number и выше
List<? super Integer> integers;  // запись Integer и ниже
```

# JAVA: АННОТАЦИИ

```
// Встроенные аннотации
@Override  // переопределение метода
@SuppressWarnings("unchecked")  // подавление предупреждений
@Deprecated  // устаревший элемент

// Кастомные аннотации
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
public @interface MyAnnotation {
    String value() default "";
    int priority() default 0;
}

// Использование
@MyAnnotation(value = "test", priority = 1)
public void myMethod() { }

// Чтение аннотаций через Reflection
MyAnnotation ann = myMethod.getAnnotation(MyAnnotation.class);
```

```
System.out.println(ann.value());  // "test"
```

# JAVA: ENUMERATIONS

```
// Простой enum
enum Day {
    MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY, SU
}

// Enum с полями и методами
enum Planet {
    MERCURY(3.303e+23, 2.4397e6),
    VENUS(4.869e+24, 6.0518e6),
    EARTH(5.976e+24, 6.37814e6);

    private final double mass;
    private final double radius;

    Planet(double mass, double radius) {
        this.mass = mass;
        this.radius = radius;
    }
```

```
    double surfaceGravity() {
        final double G = 6.67300E-11;
        return G * mass / (radius * radius);
    }
}

// Switch c enum (Java 14+)
String desc = switch (day) {
    case MONDAY, FRIDAY -> "Start of week";
    case SATURDAY, SUNDAY -> "Weekend";
    default -> "Midweek";
};
```

# JAVA: ВНУТРЕННИЕ КЛАССЫ

## MEMBER INNER CLASS

```
class Outer {
    private int x = 10;
    class Inner {
        void show() {
            System.out.println(x);
        }
    }
}
new Outer().new Inner().show();
```

## STATIC NESTED CLASS

```
class Outer {
    static int x = 10;
    static class Nested {
        void show() {
            System.out.println(x);
        }
    }
}
new Outer.Nested().show();
```

## LOCAL CLASS

```
void method() {
    class Local {
        void show() {
            System.out.println("local");
        }
    }
    new Local().show();
}
```

## ANONYMOUS CLASS

```
Runnable r = new Runnable() {
    @Override
    public void run() {
        System.out.println("anon");
    }
};
// Или с лямбдой (functional interface)
Runnable r2 = () ->
    System.out.println("lambda");
```

# JAVA: ОБРАБОТКА ИСКЛЮЧЕНИЙ

```
// Иерархия исключений
// Throwable
//    ├── Error (необрабатываемые: OutOfMemoryError, StackOverf
//    └── Exception
//        ├── Checked Exceptions (IOException, SQLException)
//        └── RuntimeException (NullPointerException, ArrayInde

// Try-catch-finally
try {
    int result = 10 / 0;
} catch (ArithmeticException e) {
    System.err.println("Ошибка: " + e.getMessage());
} finally {
    System.out.println("Всегда выполняется");
}

// Try-with-resources (AutoCloseable)
try (var reader = new BufferedReader(new FileReader("file.txt"
    String line = reader.readLine();
```

```
}  // reader.close() вызывается автоматически

// Multi-catch (Java 7+)
try {
    // ...
} catch (IOException | SQLException e) {
    // обработка обоих типов
}
```

# JAVA: COLLECTIONS FRAMEWORK

## COLLECTION ИЕРАРХИЯ

- **List** (порядок, дубликаты)
  - ArrayList - массив
  - LinkedList - связный список
  - Vector - синхронизированный
- **Set** (без дубликатов)
  - HashSet - хэш-таблица
  - LinkedHashSet - сохраняет порядок
  - TreeSet - отсортированный

## MAP (КЛЮЧ-ЗНАЧЕНИЕ)

- **HashMap** - хэш-таблица
- **LinkedHashMap** - сохраняет порядок
- **TreeMap** - отсортированный по ключу
- **Hashtable** - синхронизированный

## QUEUE / DEQUE

- **PriorityQueue**
- **ArrayDeque**

# JAVA: STREAM API

```
List<String> names = Arrays.asList("Alice", "Bob", "Charlie",

// Фильтрация, сортировка, преобразование
List<String> result = names.stream()
    .filter(name -> name.length() > 3)        // фильтрация
    .map(String::toUpperCase)                  // преобразован
    .sorted()                                  // сортировка
    .collect(Collectors.toList());             // сбор в List

// Параллельный stream
long count = names.parallelStream()
    .filter(name -> name.startsWith("A"))
    .count();

// Агрегатные операции
Optional<String> longest = names.stream()
    .max(Comparator.comparingInt(String::length));

// Группировка
```

```
Map<Integer, List<String>> byLength = names.stream()
    .collect(Collectors.groupingBy(String::length));

//reduce
int sum = IntStream.range(1, 101).reduce(0, Integer::sum);
```

# JAVA: ФУНКЦИОНАЛЬНЫЕ ИНТЕРФЕЙСЫ

```
// Функциональный интерфейс - интерфейс с одним abstract метод
@FunctionalInterface
interface Transformer<T, R> {
    R transform(T input);
}

// Использование с лямбдой
Transformer<String, Integer> length = String::length;
System.out.println(length.transform("Hello"));  // 5

// Стандартные функциональные интерфейсы (java.util.function)
Predicate<String> isLong = s -> s.length() > 5;
Function<String, Integer> toLen = String::length;
Consumer<String> print = System.out::println;
Supplier<String> greeting = () -> "Hello";
UnaryOperator<String> upper = String::toUpperCase;
BinaryOperator<Integer> sum = Integer::sum;
```

```
// Цепочки
List<String> result = names.stream()
    .filter(isLong)                 // Predicate
    .map(upper)                     // UnaryOperator
    .toList();
```

# KOTLIN: СОВРЕМЕННЫЙ ЯЗЫК ДЛЯ ANDROID

# KOTLIN: ЗАЧЕМ НУЖЕН?

- Google объявил Kotlin **предпочтительным языком** для Android (2019)
- К 2025 году ~95% новых Android-приложений пишутся на Kotlin
- **100% совместимость** с Java
- Короче, безопаснее, выразительнее
- Поддержка Android Studio "из коробки"

# KOTLIN VS JAVA: ОСНОВНЫЕ ОТЛИЧИЯ

| Особенность | Java | Kotlin |
|---|---|---|
| Null-safety | NullPointerException | Тип nullable (?T) |
| Data classes | getters/setters/equals/hashCode вручную | data class |
| Lambda | (a, b) -> a + b | { a, b -> a + b } |
| Extension functions | Нет | fun String.removeFirst() |
| Coroutines | Thread, ExecutorService | suspend fun, launch {} |
| String templates | "Hello " + name | "Hello $name" |
| Smart casts | instanceof + каст | автоматический каст |

# KOTLIN: NULL-SAFETY

```
// Java: NullPointerException - основная причина крашей
String name = null;
System.out.println(name.length()); // Краш!

// Kotlin: компилятор не даст скомпилировать
var name: String = "Hello"
// name = null  // Ошибка компиляции!

var nullableName: String? = "Hello"
nullableName = null
println(nullableName?.length)  // null (безопасный вызов)
println(nullableName?.length ?: 0)  // 0 (элвис-оператор)
```

# KOTLIN: EXTENSION FUNCTIONS

```
// Расширение стандартного класса
fun String.removeFirst(): String {
    return this.substring(1)
}

println("Hello".removeFirst())  // "ello"

// Расширение для TextView
fun TextView.showToast(text: String) {
    Toast.makeText(this.context, text, Toast.LENGTH_SHORT)
}

myTextView.showToast("Привет!")
```

# KOTLIN: COROUTINES

```
// Корутины - лёгковесные потоки для асинхронного кода
lifecycleScope.launch {
    val data = fetchDataFromNetwork()  // suspend fun
    updateUI(data)
}

// Определение suspend-функции
suspend fun fetchDataFromNetwork(): String {
    return withContext(Dispatchers.IO) {
        val url = URL("https://api.example.com/data")
        url.readText()
    }
}
```

- **Dispatchers.IO** - сетевые/файловые операции
- **Dispatchers.Main** - обновление UI
- **Dispatchers.Default** - CPU-интенсивные задачи

# KOTLIN: DATA CLASSES

```
// Java: 50+ строк кода для аналогичного класса
data class User(
    val id: Long,
    val name: String,
    val email: String,
    val age: Int = 0  // значение по умолчанию
)

// Автоматически генерируются:
// equals(), hashCode(), toString(), copy(), componentN()

val user1 = User(1, "Иван", "ivan@mail.ru")
val user2 = user1.copy(name = "Пётр")
println(user1)  // User(id=1, name=Иван, email=ivan@mail.r
```

# KOTLIN: WHEN EXPRESSION

```
// When как замена switch
val x = 1
val description = when (x) {
    1 -> "один"
    2 -> "два"
    3, 4 -> "три или четыре"
    in 5..10 -> "от пяти до десяти"
    !in 11..20 -> "не от 11 до 20"
    else -> "другое"
}

// When без аргумента (как if-else chain)
val temperature = 25
when {
    temperature < 0 -> println("Мороз")
    temperature in 0..15 -> println("Прохладно")
    temperature in 16..30 -> println("Тепло")
    else -> println("Жарко")
}
```

```
// When для проверки типа
when (obj) {
    is String -> println(obj.length)
    is Int -> println(obj + 1)
    is List<*> -> println(obj.size)
}
```

# KOTLIN: SEALED CLASSES

```
// Sealed class - ограниченная иерархия наследования
sealed class Result<out T> {
    data class Success<T>(val data: T) : Result<T>()
    data class Error(val exception: Exception) : Result<Nothin
    object Loading : Result<Nothing>()
}

// When exhaustive - компилятор проверяет все ветки
fun handleResult(result: Result<String>) = when (result) {
    is Result.Success -> println("Данные: ${result.data}")
    is Result.Error -> println("Ошибка: ${result.exception}")
    Result.Loading -> println("Загрузка...")
    // Не нужен else - все варианты покрыты
}

// Sealed interface (Kotlin 1.5+)
sealed interface UIState {
    data object Idle : UIState
    data class Loaded(val items: List<String>) : UIState
```

```
    data class Error(val msg: String) : UIState
}
```

# KOTLIN: INLINE FUNCTIONS

```
// Inline function - тело вставляется в место вызова
inline fun measureTime(block: () -> Unit): Long {
    val start = System.nanoTime()
    block()
    return System.nanoTime() - start
}

val elapsed = measureTime {
    // этот код будет вставлен напрямую
    Thread.sleep(100)
}

// noinline - не инлайнить конкретные параметры
inline fun combine(
    noinline action1: () -> Unit,
    noinline action2: () -> Unit
) {
    action1()
    action2()
```

```
}

// Reified type parameters
inline fun <reified T> Any.isInstanceOf(): Boolean {
    return this is T
}

println("hello".isInstanceOf<String>())  // true
```

# KOTLIN: SCOPE FUNCTIONS (LET/RUN/WITH/APPLY/ALSO)

| Функция | Объект (this/it) | Возврат | Пример |
|---|---|---|---|
| **let** | it | Lambda | obj?.let { it.doSomething() } |
| **run** | this | Lambda | obj.run { this.name = "X" } |
| **with** | this | Lambda | with(obj) { print(name) } |
| **apply** | this | Объект | obj.apply { name = "X" } |
| **also** | it | Объект | obj.also { it.log() } |

```
// let - безопасный вызов на nullable
val length: Int? = nullableString?.let {
    println("Длина: ${it.length}")
    it.length
}

// apply - конфигурация объекта
val textView = TextView(context).apply {
```

```
    text = "Hello"
    textSize = 16f
    setTextColor(Color.BLACK)
}

// also - побочные действия
val numbers = mutableListOf(1, 2, 3).also {
    println("Список: $it")
}
```

# KOTLIN: HIGHER-ORDER FUNCTIONS

```
// Функция, принимающая функции как параметры
fun operate(a: Int, b: Int, operation: (Int, Int) -> Int): Int
    return operation(a, b)
}

val sum = operate(5, 3) { x, y -> x + y }  // 8
val product = operate(5, 3) { x, y -> x * y }  // 15

// Функция, возвращающая функцию
fun getGreeting(language: String): (String) -> String {
    return when (language) {
        "ru" -> { name -> "Привет, $name!" }
        "en" -> { name -> "Hello, $name!" }
        else -> { name -> "Hi, $name!" }
    }
}

val greetRu = getGreeting("ru")
println(greetRu("Иван"))  // Привет, Иван!
```

```
// Функция высшего порядка с блоком
inline fun <T> T.also(block: (T) -> Unit): T {
    block(this)
    return this
}
```

# KOTLIN: DELEGATED PROPERTIES

```
// Делегированные свойства - делегирование логики доступа
class Example {
    var p: String by Delegate()
}

class Delegate {
    operator fun getValue(thisRef: Any?, property: KProperty<*
        return "$thisRef, делегат для '${property.name}'"
    }
    operator fun setValue(thisRef: Any?, property: KProperty<*
        println("$thisRef -> ${property.name} = $value")
    }
}

// Встроенные делегаты
class User {
    var name: String by observable("Нет имени") { _, old, new
        println("$old -> $new")
    }
```

```
    var age: Int by vetoable(0) { _, _, new ->
        new >= 0  // отрицательный возраст отклоняется
    }

    val lazyValue: String by lazy {
        println("Вычисление...")
        "Готово"
    }
}

// Android-примеры
// by viewBinding() - делегат привязки View
// by navArgs() - делегат аргументов навигации
```

# KOTLIN: FLOW BASICS

```
import kotlinx.coroutines.flow.*

// Flow - холодный асинхронный поток данных
fun numberFlow(): Flow<Int> = flow {
    for (i in 1..5) {
        delay(100)  // эмуляция асинхронной операции
        emit(i)     // отправка значения
    }
}

// Запуск потока (-collect)
lifecycleScope.launch {
    numberFlow()
        .filter { it % 2 == 0 }      // только чётные
        .map { it * it }              // возведение в квадрат
        .collect { println(it) }      // 4, 16
}

// StateFlow для состояния (замена LiveData)
```

```
class MyViewModel : ViewModel() {
    private val _uiState = MutableStateFlow(UiState())
    val uiState: StateFlow<UiState> = _uiState.asStateFlow()
}

// SharedFlow для событий (one-shot events)
private val _events = MutableSharedFlow<Event>()
val events: SharedFlow<Event> = _events.asSharedFlow()
```

# KOTLIN: КОГДА ИСПОЛЬЗОВАТЬ JAVA?

- Существующие проекты на Java (миграция постепенная)
- Библиотеки, написанные на Java
- Некоторые JVM-специфичные оптимизации
- В текущем курсе: **Java - основной язык, Kotlin - дополнительный**

С 2024 года Kotlin - стандарт для Android-разработки. Новые API Google пишет только на Kotlin.

# ИНСТРУМЕНТЫ РАЗРАБОТЧИКА

# ИНСТРУМЕНТЫ РАЗРАБОТЧИКА

**JDK**

Java Development Kit

**Android Studio**

Официальная IDE

**Gradle**

Система сборки

**Android SDK**

Software Development Kit

# ANDROID STUDIO

- Представлен в 2013, как замена Eclipse ADT
- Основана на IntelliJ IDEA
- Система сборки Gradle (Kotlin DSL)
- Визуальный редактор разметок (Layout Editor)
- Встроенный профайлер (CPU, Memory, Network, Energy)
- Поддержка Git, SVN
- Emulator с поддержкой скриншотов, записи, Cellular и GPS
- AI Assistant (Gemini) для генерации кода

# НАСТРОЙКА ОКРУЖЕНИЯ

1. Установить JDK 17+ (встроенный в Android Studio)
2. Установить Android Studio (Ladybug 2024.2+)
3. Настроить Android SDK через SDK Manager
4. Установить эмулятор (Pixel API 34+) или настроить реальное устройство
5. Знакомство с интерфейсом Android Studio

# ПЕРВЫЙ ЗАПУСК ANDROID STUDIO

- Создание нового проекта (Empty Activity)
- Обзор структуры проекта
- Основные файлы: build.gradle.kts, AndroidManifest.xml
- Запуск на эмуляторе
- Запуск на физическом устройстве (Developer Options + USB Debugging)

# СТРУКТУРА ANDROID-ПРОЕКТА

```
MyApp/
├── app/
│   ├── src/
│   │   ├── main/
│   │   │   ├── java/com/example/myapp/
│   │   │   │   └── MainActivity.kt
│   │   │   ├── res/
│   │   │   │   ├── layout/
│   │   │   │   │   └── activity_main.xml
│   │   │   │   ├── values/
│   │   │   │   │   ├── strings.xml
│   │   │   │   │   └── colors.xml
│   │   │   │   └── drawable/
│   │   │   └── AndroidManifest.xml
│   │   └── test/
│   └── build.gradle.kts
├── build.gradle.kts              # проектный build-файл
├── settings.gradle.kts
├── gradle.properties
```

```
└── gradlew / gradlew.bat
```

# BUILD.GRADLE.KTS

```
plugins {
    id("com.android.application")
    id("org.jetbrains.kotlin.android")
}

android {
    namespace = "com.example.myapp"
    compileSdk = 35

    defaultConfig {
        applicationId = "com.example.myapp"
        minSdk = 26
        targetSdk = 35
        versionCode = 1
        versionName = "1.0"
    }
```

```
    buildFeatures {
        compose = true  // Jetpack Compose
    }
}

dependencies {
    implementation("androidx.core:core-ktx:1.15.0")
    implementation("androidx.activity:activity-compose:1.9
    implementation(platform("androidx.compose:compose-bom:
    implementation("androidx.compose.ui:ui")
    implementation("androidx.compose.material3:material3")
}
```

# СОВРЕМЕННЫЙ ANDROID-СТЕК (2025)

## ЯЗЫК И UI

- **Kotlin** - основной язык
- **Jetpack Compose** - декларативный UI
- **Material 3** - дизайн-система
- **XML layouts** - устаревает

## АРХИТЕКТУРА

- **ViewModel** - хранение состояния
- **LiveData / Flow** - реактивное наблюдение
- **Room** - локальная БД
- **Hilt/Koin** - Dependency Injection

# JETPACK COMPOSE VS XML LAYOUTS

```
// XML Layout (устаревший подход)
<TextView
    android:id="@+id/textView"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Hello World!" />

// Jetpack Compose (современный подход)
@Composable
fun Greeting(name: String) {
    Text(
        text = "Hello $name!",
        style = MaterialTheme.typography.headlineMedium
    )
}
```

Compose: декларативный подход, меньше кода, лучшая поддержка состояния.

# ANDROID STUDIO: ГОРЯЧИЕ КЛАВИШИ

| Действие | Windows/Linux | macOS |
|---|---|---|
| Поиск класса | Ctrl+N | Cmd+O |
| Поиск файла | Ctrl+Shift+N | Cmd+Shift+O |
| Поиск по файлам | Ctrl+Shift+F | Cmd+Shift+F |
| Замена по файлам | Ctrl+Shift+R | Cmd+Shift+R |
| Рефакторинг (Rename) | Shift+F6 | Shift+F6 |
| Генерация кода | Alt+Insert | Cmd+N |
| Удаление строки | Ctrl+Y | Backspace |
| Дублирование строки | Ctrl+D | Cmd+D |
| Переход к объявлению | Ctrl+B | Cmd+B |
| Quick Fix | Alt+Enter | Alt+Enter |

# ANDROID STUDIO: LAYOUT INSPECTOR

- **Назначение:** визуальный осмотр иерархии View в реальном времени
- **Как открыть:** Tools > Layout Inspector
- **Возможности:**
  - Просмотр иерархии View (дерево компонентов)
  - Просмотр свойств каждого View (размеры, отступы, цвета)
  - Выделение View на экране кликом
  - Отображение boundaries и padding
  - Работает с Jetpack Compose (Composition tree)
- **Использование:** отладка layout, поиск скрытых View, оптимизация вложенности

# ANDROID STUDIO: DATABASE INSPECTOR

- **Назначение:** просмотр и управление локальными базами данных
- **Как открыть:** View > Tool Windows > App Inspection > Database
- **Поддерживаемые БД:**
  - Room (SQLite) - полная поддержка
  - Realm
  - Самописные SQLite
- **Возможности:**
  - Просмотр таблиц и данных
  - Выполнение SQL-запросов
  - Экспорт данных
  - Live updates при изменении данных

# ANDROID STUDIO: NETWORK INSPECTOR

- **Назначение:** мониторинг сетевых запросов приложения
- **Как открыть:** View > Tool Windows > App Inspection > Network
- **Возможности:**
  - Просмотр всех HTTP/HTTPS запросов
  - Request/Response headers и body
  - Время выполнения запросов
  - Размер ответов
  - Фильтрация по URL, коду ответа
  - Выполнение timeline-диаграммы
- **Совместимость:** OkHttp, HttpURLConnection, Retrofit

Замена Charles Proxy для базового анализа сетевого трафика.

# ANDROID STUDIO: APP INSPECTION

- **Назначение:** общий инструмент для инспекции приложений
- **Как открыть:** View > Tool Windows > App Inspection
- **Вкладки:**
  - **Database Inspector** - работа с БД
  - **Background Task Inspector** - мониторинг фоновых задач (WorkManager, AlarmManager)
  - **Network Inspector** - сетевой трафик
- **Background Task Inspector:**
  - Просмотр активных и завершённых задач
  - Детали: время, состояние, теги
  - Завершение задач вручную (для отладки)

# ПРАКТИЧЕСКИЕ РЕКОМЕНДАЦИИ

- Используйте **LTS-версии Java** (17, 21)
- Пишите на **Kotlin** для новых проектов
- Изучайте **Jetpack Compose** для UI
- Следите за обновлениями SDK
- Используйте **Android Studio** как основную IDE
- Тестируйте на реальных устройствах